



JavaScript

Урок №19: Основы. Манипуляция элементами и атрибуты.

Автор Бахшиллов Бехзод

Ментор по Frontend в **PRO** UNITY

Свойства элементов.

Node - в представлении JavaScript обычный объект, но определенного типа.

Так как **node** это обычный объект, у него есть свои свойства и методы. Расширенный список свойств и методов [здесь](#) . Самое полное описание - в [официальной спецификации](#) .

nodeType - тип узла, содержит числовое значение (только для чтения):

1 - element node

3 - text node

8 - comment node

```
var paragraph = document.querySelector('p');
```

```
paragraph.nodeType; // 1
```

tagName - содержит имя тега в верхнем регистре

nodeName - содержит имя узла (для тегов - с большой буквы)

Эти свойства только для чтения, изменить их нельзя.

```
<p>Text</p>
```

```
var paragraph = document.querySelector('p');
```

```
paragraph.tagName ; // 'P'
```

```
paragraph.nodeName ; // 'P'
```

```
paragraph.firstChild.tagName; // undefined так как текст это не тег
```

```
paragraph.firstChild.nodeName; // "#text"
```

Свойства элементов.

innerHTML - содержит html текст внутри данного элемента-узла. Это свойство можно менять (изменения произойдут сразу и на странице). Даже если вы делаете добавление к **innerHTML**

el.innerHTML += 'content',

внутреннее содержимое элемента стирается и создается заново.

```
<h1>Title with <mark>info</mark></h1>
```

```
var title = document.querySelector('h1');
```

```
title.innerHTML; // "Title with <mark>info</mark>"
```

```
title.innerHTML = 'New title';
```

```
<h1>New title</h1>
```

outerHTML - содержит html текст внутри данного узла (можно менять).

```
<h1>Title with <mark>info</mark></h1>
```

```
var title = document.querySelector('h1');
```

```
title.outerHTML; // "<h1>Title with <mark>info</mark></h1>"
```

```
title.outerHTML = '<h2>New title</h2>';
```

```
<h2>New title</h2>
```

```
title.outerHTML; // "<h1>Title with <mark>info</mark></h1>"
```

title по-прежнему ссылается на первоначальный элемент, хотя в html и визуально содержится **новый**



Свойства элементов.

data - содержит содержимое узлов для НЕ-элементов (комментарии, текст). Доступно для записи.

```
<h1>Title with <mark>info</mark></h1>  
var title = document.querySelector('h1');  
title.data; // undefined так как title - это узел-элемент (HTMLHeadingElement)  
title.firstChild.data; // 'Title with '  
title.firstChild.data = 'New text '  
<h1>New text <mark>info</mark></h1>
```

textContent - содержит текстовое содержимое узлов без тегов. Доступно для записи.

```
<h1>Title with <mark>info</mark></h1>  
var title = document.querySelector('h1');  
title.textContent; // "Title with info"  
title.textContent = 'New text';  
<h1>New text</h1>
```



Свойства. Код Задачи.

```
<div>
```

```
  <article>
```

```
    <p>Lorem ipsum dolor sit amet, odio omnesque ius cu, quo ex atqui antiopam. At detracto  
    menandri eos. Duo in causae viderer, graeci <a href="#">reprehendunt</a> has in. Decore  
    <mark>nemore</mark> philosophia te pro, nobis legere causae ex mei, odio putant mentitum  
    ea ius. Vix nostro deserunt explicari eu.</p>
```

```
  </article>
```

```
</div>
```

```
<ul>
```

```
  <li><a href="#">Link1</a></li>
```

```
  <li><a href="#">Link2</a></li>
```

```
  <li><a href="#">Link3</a></li>
```

```
  <li><a href="#">Link4</a></li>
```

```
</ul><span></span>
```

```
<a href="#">Some link</a>
```



Задачи часть 1:

1. Найти параграф и получить его текстовое содержимое (**только текст!**)
2. Создать функцию, которая принимает в качестве аргумента узел DOM и возвращает информацию (в виде объекта) о типе узла, об имени узла и о количестве дочерних узлов (если детей нет - 0).
3. Получить массив, который состоит из текстового содержимого ссылок внутри списка:
`getTextFromUl(ul) ---> ["Link1", "Link2", "Link3"]`
4. В параграфе заменить все дочерние текстовые узлы на “-text-” (**вложенные теги должны остаться**). Конечный результат:
`-text-<a href="#">reprehendunt</a>-text-<mark>nemore</mark>-text-`

Атрибуты элементов.

У тегов есть стандартные **атрибуты**. При превращении тега в узел DOM, атрибуты трансформируются в свойства узла (узел - это объект, у объекта есть методы и свойства). Это справедливо только для стандартных атрибутов

Почти всегда атрибут тега соответствует свойству узла.

```
<a href="address" id="info" class="link">Link</a>
```

```

```

```
var link = document.querySelector('a');
```

```
link.href; // вернем полный url, а не только "address" - http://site.com/address
```

```
link.className; // "link" - не просто class, а className
```

```
link.id; // "info"
```

```
var img = document.querySelector('img');
```

```
img.src; // вернем полный url, а не только "pic.gif" - http://site.com/pic.gif
```

```
img.alt; // "text"
```



Атрибуты элементов.

`element.classList` - свойство элементов, которое позволяет работать с атрибутом тега `class`.

`element.classList.contains("class")` – проверяет, есть ли у элемента `class`.

`element.classList.add/remove("class")` – добавляет/удаляет класс `class`

`element.classList.toggle("class")` – переключает `class` (добавляет/удаляет).

```
<a href="#" class="link">Link</a>
```

```
var link = document.querySelector('a');
```

```
link.classList.contains('link'); // true
```

```
link.classList.add('other'); // <a href="#" class="link other">Link</a>
```

```
link.classList.toggle('other'); // <a href="#" class="link">Link</a>
```




Атрибуты элементов.

Работать непосредственно с атрибутами тега можно с помощью свойства `attributes` и методов `getAttribute`, `setAttribute`, `removeAttribute` и `hasAttribute`.

```
<a href="address" class="link">Link</a>
```

```
var link = document.querySelector('a');
```

```
link.attributes; // NamedNodeMap {0: href, 1: class} псевдомассив
```

```
link.getAttribute('href'); // "address"
```

```
link.setAttribute('href', 'contacts'); // <a href="contacts" class="link">Link</a>
```

```
link.hasAttribute('class'); // true
```

```
link.removeAttribute('class'); // <a href="contacts">Link</a>
```



Атрибуты элементов.

В JavaScript есть специальное свойство `dataset` для работы с нестандартными атрибутами. Для этого атрибут тега должен начинаться с приставки `data-`.

```
<a href="address" class="link" data-contacts="123456">Contacts</a>  
var link = document.querySelector('a');
```

```
link.dataset.contacts; // "123456"
```



Задачи часть 2:

1. Найти в коде список `ul` и добавить класс `"list"`
2. Найти в коде ссылку, находящуюся после списка `ul`, и добавить `id=link`
3. На `li` через один (начиная с самого первого) установить класс `"item"`
4. На все ссылки в примере установить класс `"custom-link"`

Код для задач брать со слайда 5.



Создание и добавление узлов и элементов.

`document.createElement(tagName)` - создает элемент согласно заданному тегу.

Элемент создан, но ещё не отрисовался (мы его не видим).

```
var div = document.createElement('div');
```

`document.createTextNode('Some text')` - создает текстовый узел с заданным текстом.

`element.cloneNode(isDeepCopy)` - создает копию узла. Если аргументом передано `true`, копия создастся со вложенными узлами.

`parentNode.appendChild(child)` - добавит внутрь `parentNode` элемент `child`. Добавленный элемент будет вставлен после всех дочерних узлов (если узел уже был на странице, он просто будет перемещен на новое место).

```
var div = document.createElement('div');
```

```
div.className = 'new-block';
```

```
document.body.appendChild(div);
```

`parentNode.insertBefore(newElement, nextElement)` - добавит в `parentNode` элемент `newElement` перед `nextElement`.

```
var div = document.createElement('div');
```

```
document.body.insertBefore(div, document.body.firstChild);
```



Удаление узлов и элементов.

`parentNode.removeChild(child)` - удалит внутри `parentNode` элемент `child`.

Пример удаления узла:

```
var div = document.querySelector('div');  
div.parentElement.removeChild(div);
```

`parentNode.replaceChild(newChild, element)` - внутри `parentNode` заменит узел `element` на `newChild`.

`elem.remove()` - удаляет элемент. Метод не работает в IE11-

Удалить все содержимое элемента можно с помощью **innerHTML**:

```
document.querySelector('div').innerHTML = '';
```



Добавление множества узлов.

`element.insertAdjacentHTML(position, htmlElement)` - вставляет html текст в указанную позицию
position может принимать одно из следующих значений:

'beforebegin' - перед element;

'afterbegin' - внутри element, перед первым дочерним элементом (потомком);

'beforeend' - внутри element, после последнего дочернего элемента;

'afterend' - после element.

```
<!-- beforebegin -->
```

```
<p>
```

```
<!-- afterbegin -->
```

```
foo
```

```
<!-- beforeend -->
```

```
</p>
```

```
<!-- afterend -->
```

Добавление множества узлов

`element.insertAdjacentHTML(position, html)` - вставляет html текст в указанную позицию:

TEST LINK

`var strong = document.querySelector('strong');`

`strong.insertAdjacentHTML('beforeend', '<span>Span</span>');`

TEST LINK

Span

`element.insertAdjacentElement(position, element)` - вставляет element в указанную позицию:

****TEST LINK****

Span

`var strong = document.querySelector('strong');`

`span = document.querySelector('span');`

`strong.insertAdjacentElement('beforebegin', span);`

Span

****TEST LINK****

Добавление множества узлов

`element.insertAdjacentText(position, text)` - вставляет текст в указанную позицию:

```
<strong>TEST <a href="home">LINK</a></strong>
```

```
var strong = document.querySelector('strong'),  
newText = 'Test string';  
strong.insertAdjacentText('beforeend', newText);
```

```
<strong>TEST <a href="home">LINK</a>Test string</strong>
```

`document.createDocumentFragment()` - метод создает объект, похожий на узел DOM (но это не node!). У него есть методы `appendChild`, `removeChild`, `children` и другие, но нет таких, как `innerHTML` и `tagName`.

При добавлении `documentFragment` на страницу, добавляются только его дети.

```
<section>Test</section>
```

```
var fragment = document.createDocumentFragment(),  
div = document.createElement('div');  
fragment.appendChild(div);  
document.querySelector('section').appendChild(fragment);
```

```
<section>Test<div></div></section>
```




Задачи часть 3:

1. Не используя innerHTML, добавить в список несколько li с классом 'new-item' и текстом 'item' + номер li:

```
<ul>
```

```
</li><a href="#">Link1</a></li>
```

```
...
```

```
<li class="new-item">item 5</li>
```

```
<li class="new-item">item 6</li>
```

```
</ul>
```

Вручную номер li не ставить оно должно подставляться в зависимости от кол-ва лишек в списке.

Код для задач брать со слайда 4.



Задачи часть 3:

2. В каждую ссылку, которая находится внутри списка `ul` добавить по тегу `strong` (в каждую ссылку один - `strong`).
3. В начало документа (в начало `body`) добавить картинку `img` с атрибутами `src` и `alt` (текст придумайте сами). В `src` добавьте реальный url к картинке. Для создания элемента используйте метод `createElement`.
4. Найти на странице элемент `mark`, добавить в конец содержимого текст `"green"` и на элемент установить класс `green`
5. Отсортировать `li` внутри списка в обратном порядке (по тексту внутри)

Код для задач брать со слайда 4.

Задачи часть 4:

6. ВАЖНО. ДАННОЕ ЗАДАНИЕ ВЫПОЛНЯЙТЕ ПОСЛЕ ПРОЕКТА TODO.

Дан массив пользователей, его можно скопировать [отсюда](#) из первой задачи, создать таблицу вида:

#	Name	Email	Balance
1	Buckner Osborne	bucknerosborne@empirica.com	2853.33
2	Rosalie Smith	rosaliesmith@katakana.com	1464.63
3	Estrada Davenport	estradaavenport@ebidco.com	2823.39
			Total balance: 7141.35

Условия:

- В конце таблицы обязательно последняя tr должна содержать total balance всех пользователей из таблицы при этом он должен быть всегда выровнен по правому краю.
- Количество пользователей может быть любым.
- Таблица и все ее содержимое должно создаваться через js, в разметке у вас может быть только контейнер.
- В коде у вас должна быть переменная которая будет содержать в виде объекта список полей и заголовков th которые будут выводиться в таблице. Что то типа { name: 'Name', email: 'Email'... } соответственно ключ объекта это ваше поле которое вы хотите вывести из объекта пользователя а значение это заголовок th.